

Heurística GA y Heurística Híbrida

Oscar Díaz

Universidad EAFIT

22 de mayo de 2026

1 Introducción

- Descripción del Problema
- Descripción algoritmos
 - BRKGA — Biased Random-Key Genetic Algorithm
 - VND — Variable Neighborhood Descent
 - EPR — Evolutionary Path Relinking + VND

2 Metodología

3 Resultados

- BRKGA y EPR respecto a la cota inferior:
- Experimentos BRKGA:
- Experimentos EPR:

4 Discusión

- Análisis de Resultados
- Conclusiones

5 Referencias

Descripción del Problema

El **No-Wait Job Shop Scheduling Problem (NWJSSP)** consiste en programar un conjunto de n trabajos sobre un conjunto de m máquinas, minimizando el tiempo de flujo total $Z = \sum_j C_j$, donde C_j es el tiempo de finalización del trabajo j .

Cada trabajo j posee:

- Una **fecha de liberación** r_j : instante más temprano en que puede comenzar a procesarse.
- Una **secuencia fija de m operaciones**, donde la operación u requiere la máquina m_{ju} con un tiempo de procesamiento p_{ju} .

Las restricciones del problema son:

- 1 Cada operación u de un trabajo solo puede iniciar tras completar la operación $u - 1$.
- 2 Una vez iniciada, ninguna operación puede interrumpirse.
- 3 Cada máquina procesa a lo sumo un trabajo a la vez.
- 4 **Restricción No-Wait**: no se permite tiempo de espera entre operaciones consecutivas del mismo trabajo

El **Biased Random-Key Genetic Algorithm (BRKGA)** es un algoritmo evolutivo que codifica las soluciones mediante **Random Keys**: vectores de reales en $[0, 1)^n$ cuyo orden relativo define una permutación de trabajos.

Ventaja de la codificación: cualquier vector en $[0, 1)^n$ representa automáticamente una permutación válida, lo que elimina la necesidad de una función de reparación.

Clase Individual:

- **keys**: lista de n valores aleatorios en $[0, 1)$.
- **sequence**: permutación obtenida al ordenar los índices de trabajos por su *key* asociada (ascendente).
- **objective**: flujo total $Z = \sum_j C_j$ (evaluación *lazy*, calculada solo si aún no se ha evaluado).

Parámetros de la población:

- Tamaño de población: `POP_SIZE` = 100
- Tamaño de élite: `ELITE_SIZE` = 25

Selección por torneo (`tournament_selection`): Se eligen $\text{TOURNAMENT_K} = 2$ individuos al azar de la población evaluada y se retorna el de menor objetivo.

Cruce sesgado (`bias_crossover`): Para cada gen i , el hijo hereda la *key* del **mejor padre** con probabilidad $\text{BIAS_FATHER} = 0,75$, y del otro padre con probabilidad $0,25$. Lo que teóricamente concentra la búsqueda hacia regiones prometedoras.

Mutación gaussiana (`mutate`):

- Se activa con probabilidad $\text{MUT_PROB} = 0,20$.
- Perturba $\text{SIZE_MUTATE_KEYS} = 15\%$ de los genes (mínimo 1) con ruido $\mathcal{N}(0, 0,15)$.
- Los valores resultantes se recortan al intervalo $[0, 1]$.
- La *sequence* y el *objective* se recalculan.

Procedimiento por generación:

- ➊ **Preservar élite:** copiar los ELITE_SIZE mejores individuos de la generación actual.
- ➋ **Generar offspring:** para cada uno de los $POP_SIZE - ELITE_SIZE = 75$ padres, aplicar selección por torneo para generar 75 hijos con cruce sesgado y aplicar mutación.
- ➌ **Evaluar offspring:** calcular Z para cada hijo nuevo (los individuos de élite ya están evaluados).
- ➍ **Nueva población:** unir élite + *offspring* y ordenar por objetivo ascendente.
- ➎ **Actualizar mejor global** y el contador de generaciones sin mejora.

Criterios de parada:

- Tiempo total $\geq MAX_TIME = 3600$ segundos.
- **Estancamiento:** $MAX_GEN_NO_IMPROVE = 250$ generaciones consecutivas sin mejora del mejor global.

VND — Variable Neighborhood Descent

El **Variable Neighborhood Descent (VND)** es una heurística de búsqueda local que recibe **cualquier solución inicial** y la mejora de forma sistemática explorando múltiples vecindarios en secuencia.

Vecindarios utilizados (en orden):

- N_1 — **Insertion Down**: extraer el trabajo de la posición i e insertarlo en posición $j > i$.
- N_2 — **Swap**: intercambiar los trabajos de las posiciones i y j (con $i < j$).
- N_3 — **Insertion Up**: extraer el trabajo de la posición i e insertarlo en posición $j < i$.

Criterio de aceptación: First Improvement — se acepta el primer vecino que mejore la solución actual, sin explorar el resto del vecindario.

Flujo del algoritmo (vnd):

- Inicializar $k = 0$ (primer vecindario: Insertion Down).
- Aplicar **First Improvement** sobre N_k (`first_improvement_ls`):
 - Generar todos los vecinos del vecindario N_k .
 - Evaluar cada vecino con `evaluate_sequence_preciso`.
 - Al encontrar $Z(\text{vecino}) < Z(\text{actual})$, aceptar y detener la exploración de N_k .
- Si hubo **mejora**: actualizar solución, reiniciar $k = 0$.
- Si **no hubo mejora**: avanzar $k \leftarrow k + 1$.
- **Parar** cuando $k = 3$ (todos los vecindarios agotados sin mejora) o al superar el límite de tiempo `time_limit`.

EPR — Evolutionary Path Relinking + VND

El **EPR (Evolutionary Path Relinking)** es el algoritmo híbrido. Combina **tres componentes** en un único flujo integrado:

- 1 **BRKGA** — componente evolutivo con *Random Keys* que genera y evoluciona la población (ver sección anterior).
- 2 **Path Relinking (PR)** — estrategia de intensificación que explora el camino entre dos soluciones del **pool de élite**, selecciona una solución intermedia y la mejora.
- 3 **VND** — búsqueda local sistemática aplicada sobre la solución intermedia seleccionada (ver sección anterior).

Idea central: el BRKGA diversifica y mantiene la población; el Path Relinking intensifica combinando pares de soluciones de alta calidad; el VND refina localmente la solución resultada del PR.

El **pool de elite** (ElitePool) mantiene las `ELITE_POOL_SIZE = 5` mejores soluciones distintas encontradas durante toda la ejecución.

Gestión del pool:

- Al agregar una solución: se verifica que no sea duplicado exacto.
- Si el pool está lleno, se reemplaza la **peor solución** solo si la nueva mejora su objetivo.
- Las soluciones se mantienen ordenadas de mejor a peor.

Uso en EPR:

- Cada `ELITE_SIZE = 25` mejores individuos de cada generación se añaden al pool.
- El método `get_pair()` selecciona aleatoriamente dos soluciones distintas del pool para usarlas como (**source, target**) en el Path Relinking.

Distancia entre secuencias (*position_distance*): Para medir cuán distintas son dos permutaciones, se calcula la suma de diferencias absolutas de posición de cada trabajo:

$$d(A, B) = \sum_{j=0}^{n-1} |\text{pos}_A(j) - \text{pos}_B(j)|$$

Movimiento dirigido (*path_step*): Dado el par (*current*, *target*), se aplica **un único movimiento** que acerca *current* a *target*:

- Para cada trabajo cuya posición difiera entre ambas secuencias, se simula su traslado a la posición que ocupa en *target*.
- Se elige el trabajo cuyo movimiento produce la **mayor reducción de distancia**.
- Este movimiento garantiza **convergencia** (la distancia disminuye estrictamente en cada paso) y es reproducible.

Flujo de la estrategia PR + VND:

- 1 **Recorrer el camino completo** de *source* a *target* aplicando `path_step` repetidamente. Se guardan **todas las soluciones intermedias** generadas (excluyendo *source* y *target*).
- 2 **Seleccionar aleatoriamente** una solución intermedia del camino. Esta elección introduce **diversificación**: distintas ejecuciones del PR entre el mismo par exploran regiones distintas.
- 3 **Aplicar VND** sobre la solución intermedia seleccionada, con presupuesto de tiempo `VND_TIME_LIMIT = 60` segundos.
- 4 **Retornar la mejor** entre: $VND(\text{intermedia})$, *source* y *target*.

Nota: si no queda tiempo suficiente, se retorna la solución intermedia sin aplicar VND.

Flujo completo del algoritmo epr:

- ❶ **Inicialización:** crear población BRKGA de $POP_SIZE = 100$ individuos, evaluarlos y poblar el `ElitePool`.
- ❷ **Ciclo generacional** (mientras haya tiempo):
 - Preservar élite + generar y evaluar *offspring* (cruce sesgado + mutación gaussiana).
 - Actualizar pool con los mejores `ELITE_SIZE` de la generación.
 - Actualizar mejor global y contador de estancamiento.
- ❸ **Path Relinking periódico** (cada $EPR_FREQ = 25$ generaciones):
 - Solo si $pool \geq 2$ soluciones y quedan > 30 segundos disponibles.
 - Ejecutar `path_relinking_with_vnd` sobre un par aleatorio del pool.
 - Si la solución PR mejora el mejor global, actualizarlo.
 - **Inyectar** la solución PR en la población (reemplaza al peor individuo).
- ❹ **Parar** por tiempo ($MAX_TIME = 3600$ s) o estancamiento ($MAX_GEN_NO_IMPROVE = 150$ generaciones).

Cuadro: Parámetros del algoritmo

Parámetro	Variable	Valor
Tamaño de población	POP_SIZE	100
Tamaño de élite	ELITE_SIZE	25
Probabilidad mutación	MUT_PROB	0.20
Sesgo cruce	BIAS_FATHER	0.75
Genes mutados	SIZE_MUTATE_KEYS	15 %
Torneo	TOURNAMENT_K	2
Gen. sin mejora	MAX_GEN_NO_IMPROVE	150
Frecuencia PR	EPR_FREQ	25
Pool de élite	ELITE_POOL_SIZE	5
Tiempo límite VND	VND_TIME_LIMIT	60 s
Tiempo total	MAX_TIME	3600 s

- ➊ **Implementación de algoritmos en Python:** Se programaron dos algoritmos genéticos, ambos aplicados al NWJSSP.
- ➋ **Ejecución sobre instancias y exportación de resultados:** Cada uno de los dos scripts recorre automáticamente las primeras 14 instancias disponibles y exporta los resultados en el formato requerido a archivos Excel separados (uno por algoritmo).
- ➌ **Uso de cota inferior proporcionada:** Se utilizan directamente las cotas inferiores dadas por el profesor en el archivo `lb.txt` para calcular el GAP de cada solución.
- ➍ **Comparación y análisis de resultados:** Se comparan los dos algoritmos frente a la cota inferior y se reportan los resultados

Exposición de Resultados

Los resultados se presentan en el siguiente orden. Primero se muestra la **cota inferior teórica** (LB) utilizada para cada instancia, que sirve como referencia para medir la calidad de las soluciones. Luego se presentan los **resultados principales** de los ocho algoritmos, valor de la función objetivo Z y tiempo de cómputo t_c (en ms), junto con el **GAP** relativo respecto a la cota inferior:

$$\text{GAP} = \frac{Z - LB}{LB}$$

Finalmente se presentan los **experimentos de parámetros** fueron realizados para las primeras seis y ocho instancias por coste computacional: para cada método de búsqueda local se varía el parámetro MIXED R, con el objetivo de identificar la configuración que produce los mejores valores de Z . Mientras que para el metaheurístico híbrido se cambian diferentes parámetros.

Cotas inferiores

Índice	Instancia	n	m	LB
1	ft06	6	6	259
2	ft06r	6	6	283
3	ft10	10	10	7377
4	ft10r	10	10	6318
5	ft20	20	5	14139
6	ft20r	20	5	13257
7	tai_j10_m10_1	10	10	64463
8	tai_j10_m10_1r	10	10	61293
9	tai_j100_m10_1	100	10	2939670
10	tai_j100_m10_1r	100	10	2827631
11	tai_j100_m100_1	100	100	5637873
12	tai_j100_m100_1r	100	100	6146561
13	tai_j1000_m10_1	1000	10	259534722
14	tai_j1000_m10_1r	1000	10	258454996

Comparación EPR vs BRKGA

Instancia	EPR	BRKGA
	GAP	GAP
1	0.189	0.189
2	0.134	0.134
3	0.330	0.340
4	0.615	0.616
5	0.195	0.220
6	0.331	0.320
7	0.460	0.526
8	0.605	0.572
9	1.183	1.146
10	1.264	1.281
11	6.862	6.775
12	6.243	6.225
13	1.238	1.215
14	1.239	1.226
Promedio	1.492	1.485

BRKGA - Variación bias_father

Instancia	bias_father=0.5	bias_father=0.75	bias_father=0.9
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.379	0.330	0.330
4	0.595	0.616	0.627
5	0.233	0.268	0.302
6	0.314	0.367	0.318
7	0.460	0.460	0.460
8	0.611	0.611	0.536
Promedio	0.364	0.372	0.362

BRKGA - Variación elite_size

Instancia	elite_size=10	elite_size=25	elite_size=40
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.330	0.330	0.361
4	0.615	0.616	0.627
5	0.232	0.268	0.217
6	0.330	0.367	0.366
7	0.460	0.460	0.460
8	0.536	0.611	0.536
Promedio	0.353	0.372	0.361

BRKGA - Variación max_gen_no_improve

Instancia	50	150	300
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.330	0.330	0.330
4	0.616	0.616	0.616
5	0.275	0.269	0.268
6	0.409	0.367	0.367
7	0.460	0.460	0.460
8	0.611	0.611	0.611
Promedio	0.378	0.372	0.372

BRKGA - Variación mut_prob

Instancia	mut_prob=0.1	mut_prob=0.2	mut_prob=0.4
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.340	0.330	0.330
4	0.627	0.616	0.615
5	0.318	0.268	0.266
6	0.368	0.367	0.320
7	0.460	0.460	0.482
8	0.536	0.611	0.611
Promedio	0.371	0.372	0.368

BRKGA - Variación pop_size

Instancia	pop_size=50	pop_size=100	pop_size=200
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.380	0.330	0.330
4	0.615	0.616	0.615
5	0.261	0.268	0.216
6	0.381	0.367	0.359
7	0.531	0.460	0.460
8	0.536	0.611	0.536
Promedio	0.378	0.372	0.355

EPR - Variación bias_father

Instancia	bias_father=0.5	bias_father=0.75	bias_father=0.9
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.379	0.330	0.330
4	0.595	0.616	0.615
5	0.200	0.260	0.300
6	0.330	0.339	0.323
7	0.460	0.460	0.460
8	0.611	0.611	0.536
Promedio	0.362	0.367	0.361

EPR - Variación elite_pool_size

Instancia	elite_pool_size=3	elite_pool_size=5	elite_pool_size=10
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.330	0.330	0.330
4	0.616	0.616	0.615
5	0.260	0.260	0.214
6	0.344	0.339	0.339
7	0.460	0.460	0.460
8	0.611	0.611	0.611
Promedio	0.368	0.367	0.361

EPR - Variación elite_size

Instancia	elite_size=10	elite_size=25	elite_size=40
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.330	0.330	0.357
4	0.615	0.616	0.615
5	0.232	0.260	0.199
6	0.346	0.339	0.321
7	0.460	0.460	0.460
8	0.536	0.611	0.536
Promedio	0.355	0.367	0.351

EPR - Variación epr_freq

Instancia	epr_freq=10	epr_freq=25	epr_freq=50
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.330	0.330	0.330
4	0.615	0.616	0.616
5	0.270	0.260	0.235
6	0.334	0.339	0.344
7	0.460	0.460	0.460
8	0.611	0.611	0.611
Promedio	0.368	0.367	0.365

EPR - Variación max_gen_no_improve

Instancia	max_gen_no_improve=50	max_gen_no_improve=150	max_gen_no_improve=300
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.330	0.330	0.330
4	0.616	0.616	0.616
5	0.260	0.260	0.218
6	0.344	0.339	0.339
7	0.460	0.460	0.460
8	0.611	0.611	0.611
Promedio	0.368	0.367	0.362

EPR - Variación mut_prob

Instancia	mut_prob=0.1	mut_prob=0.2	mut_prob=0.4
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.340	0.330	0.330
4	0.615	0.616	0.615
5	0.249	0.260	0.250
6	0.368	0.339	0.337
7	0.460	0.460	0.482
8	0.536	0.611	0.611
Promedio	0.361	0.367	0.368

EPR - Variación pop_size

Instancia	pop_size=50	pop_size=100	pop_size=200
	GAP	GAP	GAP
1	0.189	0.189	0.189
2	0.134	0.134	0.134
3	0.330	0.330	0.330
4	0.615	0.616	0.615
5	0.237	0.260	0.221
6	0.354	0.339	0.331
7	0.531	0.460	0.460
8	0.536	0.611	0.536
Promedio	0.366	0.367	0.352

- El algoritmo **BRKGA** obtuvo un mejor GAP promedio que el algoritmo híbrido **EPR**, aunque la diferencia fue relativamente pequeña.
- Esta ventaja del BRKGA se hizo más evidente en las instancias de mayor tamaño (especialmente en las instancias de 100 y 1000 trabajos), donde el algoritmo evolutivo puro mostró mejor escalabilidad.
- El EPR, a pesar de incorporar intensificación mediante Path Relinking y VND, no logró superar consistentemente al BRKGA dentro del límite de tiempo establecido.

- Los parámetros relacionados con la **aleatoriedad** (`bias_father` y `mut_prob`) mostraron que valores más aleatorios tienden a ser más efectivos:
 - `bias_father` = 0.5 (menor sesgo) obtuvo mejores resultados que valores más altos (0.75 y 0.9).
 - Una mayor probabilidad de mutación también fue beneficiosa.
- Los parámetros `pop_size` y `max_gen_no_improve` respondieron positivamente a su aumento, permitiendo una exploración más amplia y mayor oportunidad de escapar de óptimos locales.
- El parámetro `elite_size` presentó un comportamiento irregular, con buenos resultados tanto en valores bajos (10) como altos (40), posiblemente influenciado por la aleatoriedad inherente del algoritmo.

- Los parámetros compartidos con BRKGA (`bias_father`, `pop_size`, `elite_size`, etc.) mostraron comportamientos similares, a **excepción de** `mut_prob`:
 - En EPR, un aumento en la probabilidad de mutación tendió a **empeorar** los resultados. Esto sugiere que mutar demasiado las soluciones hijas afecta negativamente la calidad de las soluciones que luego se intensifican con Path Relinking + VND.
- El parámetro `elite_pool_size` mejoró consistentemente al aumentar su valor, lo que refuerza la importancia de mantener un pool más diverso de soluciones élite para el Path Relinking.
- El parámetro `epr_freq` mostró un resultado contraintuitivo pero interesante: mejores resultados se obtuvieron al ejecutar Path Relinking con menor frecuencia (cada 50 generaciones en lugar de cada 10). Esto sugiere que una intensificación excesiva puede ser contraproducente.

- La **aleatoriedad controlada** parece ser más efectiva que los mecanismos fuertemente deterministas o de intensificación agresiva en este problema (NWJSSP con restricción No-Wait).
- El algoritmo BRKGA puro demostró ser con mejor solución respecto al GAP que su versión híbrida (EPR) bajo las condiciones de tiempo evaluadas.
- En el EPR, es clave encontrar un buen equilibrio entre diversificación y intensificación. Una frecuencia muy alta de Path Relinking y una mutación excesiva pueden deteriorar el desempeño.
- En general comparando los resultados con los de los demás trabajos, el desempeño de ambos algoritmos es el mejor respecto a los algoritmos deterministas.

Referencias